

Композиционные обобщители в языковых моделях

Композиционные обобщители могут привести к композиционному обобщению: мы наблюдаем свойство, характерное для обучения языковых моделей, при котором задачи со схожей структурой рассматриваются как изоморфные, а все отдельные вызовы языковой модели в системе становятся взаимосвязанными.

АВТОРЫ
Алекс Чжан
Омар Хаттаб

ОПУБЛИКОВАНО
20 июля 2026 г.

ОРГАНИЗАЦИИ
MIT CSAIL
MIT CSAIL

Современное пост-обучение превратилось в грубую силу, направленную на создание все большего количества сред и увеличение сроков обучения. Во многом это связано с тем, что передовые трансформеры все еще плохо справляются с *композиционным обобщением* — способностью решать новые задачи, комбинируя уже знакомые. Если наши модели не будут объединять отдельные уроки, которые они усваивают, масштабирование будет приносить меньшую отдачу, чем могло бы, поскольку каждая новая область потребует собственных инвестиций в виде обучающих данных.

Конечно, обучающие данные — не единственный рычаг воздействия. В последние несколько лет мы решали более сложные задачи, *дополняя* трансформер, сначала логическими рассуждениями, затем использованием инструментов и так далее. Однако во всех этих случаях *генерализация* оставалась на усмотрение базовой нейронной сети и ее *индуктивных предубеждений на уровне токенов, появившихся в 2017 году*. Мы считаем, что улучшение генерализации — это во многом задача того, что сегодня называют *инфраструктурой*. Обязанность — это программа, которая находится между внешним миром и нейронной сетью: она решает, как закодировать текущее состояние среды, которое может быть сколь угодно длинным и сложным, в один или несколько входных параметров для большой языковой модели, и как определить следующее действие. **Основная задача обвязки — нести в себе индуктивный сдвиг более высокого уровня, который позволяет сводить незнакомые и сложные проблемы к более простым для базовой нейронной сети.**

Если говорить конкретно, то, по нашему мнению, хорошая «обёртка» — это такая, которая **формирует каждый вызов базового трансформера таким образом, чтобы каждое наблюдение было локально распределено в соответствии с обучающими данными**, то есть каждый вызов трансформера обрабатывает запрос, который распределен в соответствии с обучающими данными. На самом деле хорошая «обёртка» часто позволяет свести проблемы, которые, казалось бы, требуют прорыва в области пост-обучения, к практически обыденным возможностям языковых моделей текущего поколения. Впервые мы продемонстрировали эту версию для обработки длинных контекстов *почти год назад*, а в этом посте показываем, что этот принцип распространяется и на эффективность *обучения* как такового. То есть **то, что модель усваивает с помощью хорошо продуманной системы обучения, обобщается в зависимости от длины задачи и предметной области** гораздо лучше, чем при самостоятельном обучении нейронной сети.

Мы проверили это, применив обучение с подкреплением (reinforcement learning, RL) для обучения рекурсивной языковой модели (Recursive Language Model, RLM) — инструмента, в котором модель разгружает свой контекст и переносит выполнение на программную декомпозицию и рекурсивные подвызовы. Результаты представлены на рисунке 1. Обучение на коротких задачах позволяет перейти к более длинным задачам, которые в 8–32 раза длиннее, при этом прирост эффективности на этапе оценки примерно в 10 раз выше, чем при обучении базового трансформера напрямую. Более того, обучение на одном домене переносится на другие домены гораздо лучше, чем в случае с обычным трансформером.

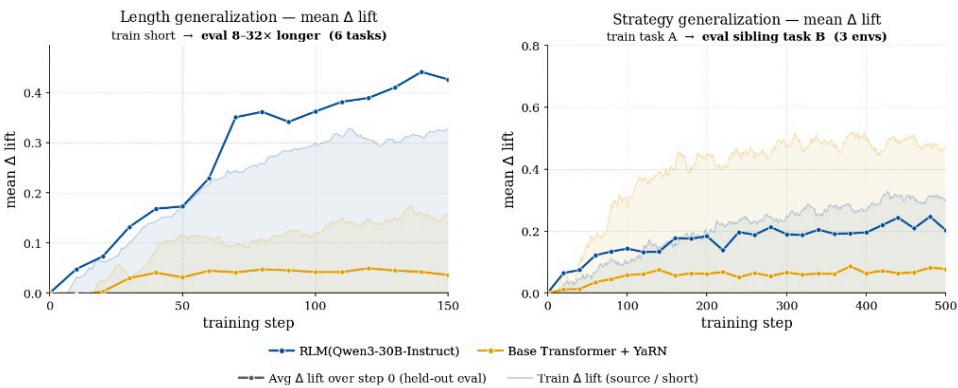


Рисунок 1. Мы обучаем Qwen3-30B-A3B-Instruct-2507 как RLM и как базовый трансформер (+ YaRN) на наборе задач, а затем строим графики обучения и оценки. Слева мы независимо обучаем 6 различных коротких моделей для решения задач и оцениваем их на тех же задачах, но с контекстом, который в 8–32 раза длиннее. Справа мы обучаем модель на 3 различных средах выполнения задач и

BrowseComp-Plus (train)

OOLong (eval)

[1] In REPL, query sub-LLM to propose decomposition based on data and query.

[1] In REPL, query sub-LLM to propose decomposition based on data and query.

[2] In parallel, fan out sub-queries to each chunk and store in memory.

[2] In parallel, fan out sub-queries to each chunk and store in memory.

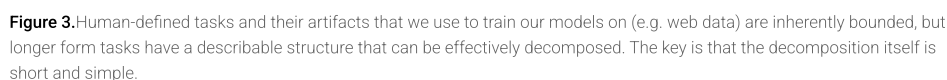
[3] Aggregate and fan in sub-call information and answer query.

[3] Aggregate and fan in sub-call information and answer query.

Root LM sees same trajectory!

Better scaling requires compositional generalization.

We argued before, via the [Mismanaged Geniuses Hypothesis \(MGH\)](#) [1], that tasks which humans care about solving, can apparently almost always be decomposed rather naturally into sub-tasks that are not only far simpler but also which are not far beyond the current generation of language models. Compositional generalization is key for leveraging this in practice and enabling our systems’ reachable task space to extend beyond their training set’s direct coverage, especially for tasks that may look nothing alike in their surface-level tokens while sharing some underlying structure.



Sadly, if the past few years of astronomic spending on training language models are any indication, Transformers and other existing neural sequence models are, at best, unreliable at compositional generalization. Though it seems very plausible that compositional generalization may emerge at a vanilla neural level, it just doesn't seem that the basic differentiable neural operations [put together 5 years after AlexNet](#) are particularly optimal when it comes to encoding the inductive biases we need for training our language systems. Indeed, precisely because we now have such powerful priors and models of language, we think it is time to seriously ask whether our inductive biases that aren't simply geometric or about simple symmetries but which can begin to live at a far higher level of abstraction.

The capacity for compositional generalization can live in the harness.

We're often interested in *agents*, which describes any loop in which some policy observes some state $\mathcal{C}$ and perform an action to interact with an environment $\mathcal{E}$. Given a state $\mathcal{C}$, it's tempting to simply serialize $\mathcal{C}$ into a prompt and tokenize that as-is for a Transformer to become our policy. But because the state and action space can be arbitrarily large and complex, we learned the hard way as a field to introduce a *harness* $H : s \rightarrow a$ which is the program that sits between the external world and the neural network: it decides how to encode the current state of the environment, which can be arbitrarily long and complex, into one or more inputs to the LLM and how to determine the next action.

Traditionally, we think of harnesses like Claude Code or Codex as necessary because of their ability to invoke external tools inside a complicated environment. But that's just part of the agent. We think that **a more fundamental power of the harness is the ability to simplify the arbitrarily complex state $\mathcal{C}$ into potentially many smaller observations $\mathcal{o}$ that each individual LM call in the harness can properly handle**. We know that Transformers are unreliable at compositional generalization, so we can rely on the harness to define higher-level inductive biases. Designing anything may trigger your bitter-lesson instincts, but there's no free lunch: it is *scalable* inductive biases that allow our neural networks to learn and generalize in the first place.

Luckily, we can put this in a way that facilitates *improved* scaling. **A good harness is a harness that reduces unfamiliar problems to familiar ones and reduces complex problems to simple ones**. In other words, even if the state $\mathcal{C}$ is out-of-distribution (OOD) to what any individual language model call was trained for, a good harness produces observations $\mathcal{o}$ that are **locally in-distribution (LID)**, which we define as every individual LM call over this observation being in-distribution with respect to the training data.

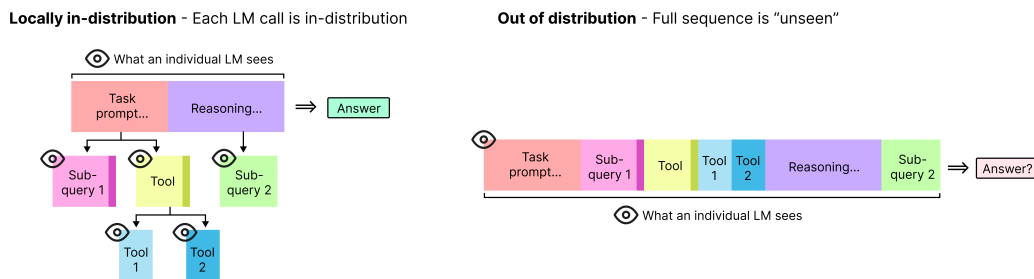


Figure 4. Good harnesses are designed so each individual LM call sees a prompt that is locally "in-distribution" with what it has learned during training, even if the full task trajectory itself is out-of-distribution (OOD).

Sadly, existing harness designs such as Claude Code and Codex fail at facilitating locally in-distribution (LID) observations for the underlying neural network. In particular, they fundamentally rely on flooding the context window of the Transformer with interleaved task-specific information, tool call outputs, and reasoning that get continuously appended. Yes, this supplies plenty of context to the model, but these bloated histories quickly fall out of the training distribution, manifesting in the ["context rot"](#) phenomenon that we often observe in practice.

Instead, we propose that a good harness should **enable individual LM calls to see structurally similar tasks as isomorphic**, enabling compositional generalization. Formally, a harness induces an equivalence operator $\sim_H$ over the set of all task states $\mathcal{T}$, meaning structurally similar tasks fall under the same harness-induced equivalence class¹ and produce similar sets of observations for the neural network involved. For most harnesses that use a main LM with sub-agents, we can understand this argument with respect to the main context, assuming the sub-agents deal with small, individually in-distribution sub-tasks.

Good harnesses induce equivalence classes over tasks, and thereby enable far better and more composable learning. A well-designed harness not only reduces the space of learnable trajectories to a small $\mathcal{T} / \sim_H$, but it also generalizes to a wider class of trajectories than the set of available training tasks, including those that do not fit in the base model's context window.

RLMs are surprisingly good at compositional generalization.

Equivalence classes over trajectories induced by the RLM harness.

An ideal harness decomposes a task such that the main context is abstracted to **literally** appear token-for-token similar for

similar problems, i.e. those that fall under the same harness-induced equivalence class. This isomorphism enables generalizations of the form: *if a system can solve task X, it should transitively be able to solve task Y*.

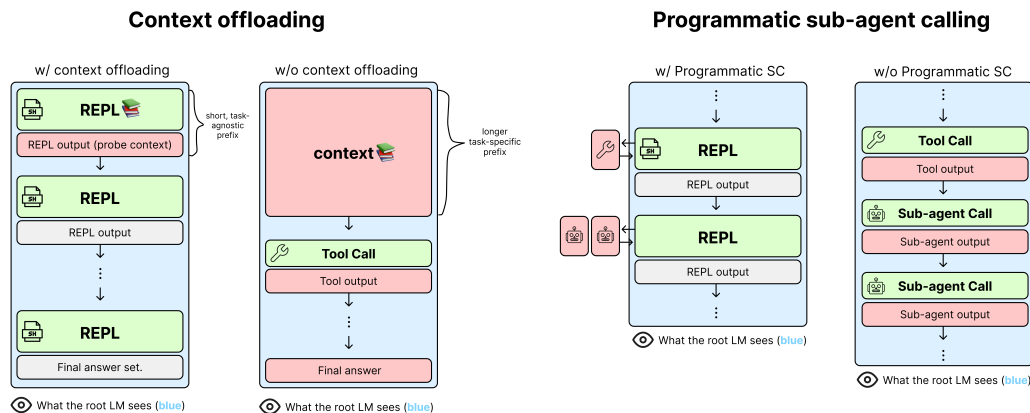


Figure 5. We compare how context offloading and programmatic sub-agent calling, two primary components of the RLM, enable the root LM to view two different problems as the same / similar. **(1)** Standard agent methods stuff the entire query and context as a prefix to the prompt. Between two different tasks, this prefix significantly changes the output distribution of the model, even if the tasks are solved the same way. Context offloading allows similar problems to be viewed as the same. **(2)** Standard agent methods use tool calls, which return information directly to the main context. Programmatic sub-agent calling allows intermediate computations and information to be stored in the REPL, so the main context does not need to see task-specific information, which can alter its output distribution.

The Recursive Language Model (RLM) harness is designed around abstracting tasks by their decomposition and deferring input-specific information to sub-calls. In this sense, RLMs view similarly decomposed problems as isomorphic², and it achieves this by:

(1) Context offloading, where input-specific context is passed as a symbolic variable so the root language model call does not directly see it, enables different problems to appear similar at the first step. Recently, several scaffolds have adopted this feature as a form of context management (see [Anthropic's Managed Agents blog](#)). On its own, context offloading does not prevent environment feedback and/or sub-agent information from returning back to the main context. Over long trajectory horizons, the main context becomes OOD, breaking LID.

(2) Programmatic sub-agent calling, where sub-agents and also normal tools are treated as functions in a code REPL, allow the root LM to selectively choose information and pass around information across further tool calls and sub-agents without the root LM itself ever needing to see it. This includes tool and sub-call outputs, which can be stored in variables in memory directly and accessed by future sub-calls. Programmatic sub-calling is equally as important as context offloading for abstracting task-specific information away from the main context.

RLMs can train on short tasks and generalize to longer, unseen tasks.

It is widely known that training Transformer-based LMs on a domain at a particular context length does not necessarily generalize to longer context lengths [2]. A large chunk of mid-training and post-training work in production models like [Owen 3.x](#), [Kimi K2.x](#), [GLM 5.x](#), etc. has been dedicated to carefully introducing longer and longer data into the training mix so LMs can learn to handle those context lengths, and then again many modern applications commonly exceed those lengths. The length generalization issue especially impacts standard agent designs like [ReAct](#) [3], [CodeAct](#) [4], [Claude Code](#), [Codex](#), etc. which as we mentioned, rely on appending observations to a growing prefix history of context.

We hypothesize that for RLMs, similar tasks of a different length can fall under the same equivalence class under $\sim_{RLM}$. In the following experiment, we explore how training only the root LM of an RLM exclusively on short sequences can generalize sequences an order of magnitude longer, as we posit that the decomposition, and hence the trajectory the RLM's root LM sees on both settings, are nearly identical.

We consider a suite of 6 different environments across different axes of length: varying the input length, output length, and number of instructions. We explicitly train only on a split containing short tasks, and evaluate on a split only containing significantly longer tasks within the same domain. Across all the settings below, we train for 150 steps (batch size 64, 4 rollouts per) with [prime-r1](#) ([Decoupled PPO](#) [5] with GRPO-like advantages + KL loss) and evaluate every 10 steps.

- [MRCRv2](#) (64k, 2-needle setting to 2M, 8-needle setting). A commonly reported benchmark in frontier model reports, the premise is to find the t th instance of needle sentence that answers some query within a large corpus of conversation data.
- [GraphWalks](#) (<128K to >1M setting). Another commonly reported benchmark in frontier model reports, the premise is to extract nodes from a graph that satisfy some simple constraint.
- [LongBenchPro](#) (32k to 256k setting, English-only). A suite of multiple choice-style questions spanning 11 different QA, code, and reasoning style problems.
- [OOLONG](#) [trec-coarse] (32k to 256k setting). A set of aggregation style questions to answer statistics about a dataset of information.
- [OOLONG-Pairs](#) (8k to 32k input, 7k to 146k output setting). A modified variant of OOLONG to find pairs of elements in the OOLONG dataset satisfying certain constraints.

- **Ada-LEval** [best-answer] (8k to 128k setting). Given a question and a large number of potential candidate and distractor answers, pick out the most suitable to answer the question.

We plot both the training reward on short tasks (translucent) and the evaluation reward (dark) on the longer version of the tasks. We compare training an RLM, an RLM where we nudge it to state the decomposition it wants to perform, and a base Transformer (with YaRN for longer settings). We use **Qwen3-30B-A3B-Instruct-2507** as the model for all settings.

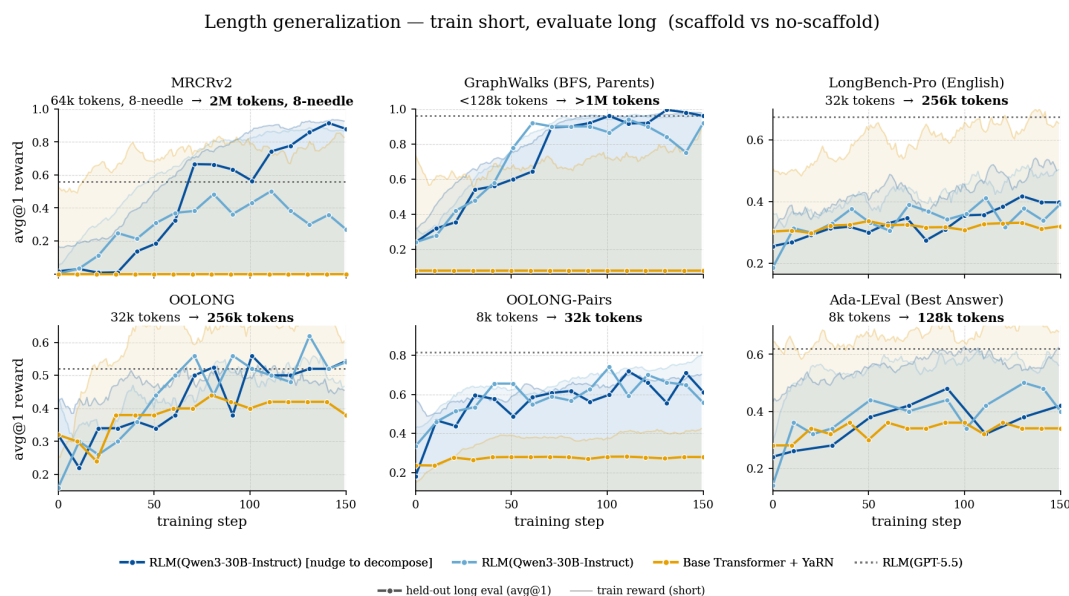


Figure 6. Across six benchmarks with length-varying splits, we train Qwen3-30B-A3B-Instruct-2507 as an RLM with a decomposition hint, a base RLM, and on its own as a Transformer, on short splits of the environment. We evaluate on longer splits 8-32x longer at every 10 step checkpoint, and plot both the evaluation score on the long eval, and train reward on short task (smoothed) over 150 steps. We also plot a frontier model, GPT-5.5, with the RLM harness as a point of comparison. The Transformer on MRCRv2 baseline was omitted because it cannot fit 2M tokens in context, even with context extension.

Across all six tasks, the RLM yields significantly better eval results on long tasks by training exclusively on short tasks, beating out the base Transformer by a significant margin even when starting with a lower step-0 eval performance. On MRCRv2, GraphWalks, OOLONG, and OOLONG-Pairs, the trained Qwen3-30B-A3B-Instruct-2507 RLM approaches or exceeds an RLM with a frontier model, GPT-5.5 on the long eval, while far exceeding the base Transformer.

RLM eval performance on longer tasks more closely matches the train reward on shorter tasks. On all six tasks, the RLM yields significant improvements in performance on the long eval relative to its starting, step-0 eval performance. The base Transformer generally yields flat eval performance despite the train reward growing and often exceeding the train reward of the RLM, suggesting *what is learned by the Transformer does not extrapolate to the longer setting*.

Why length generalization occurs and where it fails. In all of the environments above, the strategy the RLM learns to solve the short tasks is roughly equivalent to that of the strategy needed to solve the longer tasks, and because the root LM's view of the long task with the context offloaded looks near token-for-token identical to the short task, the RLM has effectively seen the same task during training. In other words, many tasks share the same equivalence class across length. The primary difference is the number of sub-calls and the content of the sub-calls themselves, each of which handle an in-distribution sub-task. Length generalization occurs because the RLM learns a generalizable strategy, *but this is not always guaranteed*. In many of the short settings above, one viable strategy is to just offload the entire problem to a single sub-call and return this as a solution, effectively becoming equivalent to the long-context Transformer baseline.

We include the “nudge to decompose” user message variant (a condensed version [of the addendum in the RLM repository](#)) to illustrate this point, which helps in the case of MRCRv2, where the RLM does not learn the right, generalizable strategy across length. In the training of RLMs and locally in-distribution LM systems as a whole, a whole area of study is how much supervision / distillation is required to get the system to converge to a generalizable solution over a non-generalizable solution. Our intuition is that at scale, no supervision is necessary, but for the sake of sample efficient learning, [some form of supervision or hint is helpful](#).

RLMs can generalize to unseen tasks that share latent structure observed in training.

The previous section showed that RLMs can learn to view tasks and their length-extrapolated variants as isomorphic by abstracting them as the same task in the root model. For length extrapolation in particular, if the RLM learns a length-agnostic strategy, it quickly generalizes to longer lengths. We can take this intuition to generalization on tasks with shared decompositions: if the RLM learns a strategy that generalizes from one domain to another (e.g. applying sorting, filter and search, MapReduce, etc.) it can similarly generalize by abstracting away the domain-specific tokens, and viewing the two problems as equivalent.

We consider 3 domains in which the context and query are referring to completely different domains, but the actual underlying strategy is roughly the same.

- **OOLONG (trac-coarse questions → spam questions).** Training on aggregation tasks about Jeopardy TREC questions and evaluating on aggregation tasks about questions whose answer is either SPAM or HAM.

- **OBLIQ-Bench Analogues (writing → math).** Training on tasks about searching for essays that seem to be written by the same author, and evaluating on tasks about searching for math problems that require the same reasoning process. Train and eval metric are nDCG@10.
- **OBLIQ-Bench Descriptive (Twitter stance → Wildchat errors).** Training on tasks about finding tweets that satisfy a certain stance, and evaluating on tasks about searching for Wildchat conversations containing errors. Train and eval metric are nDCG@10.

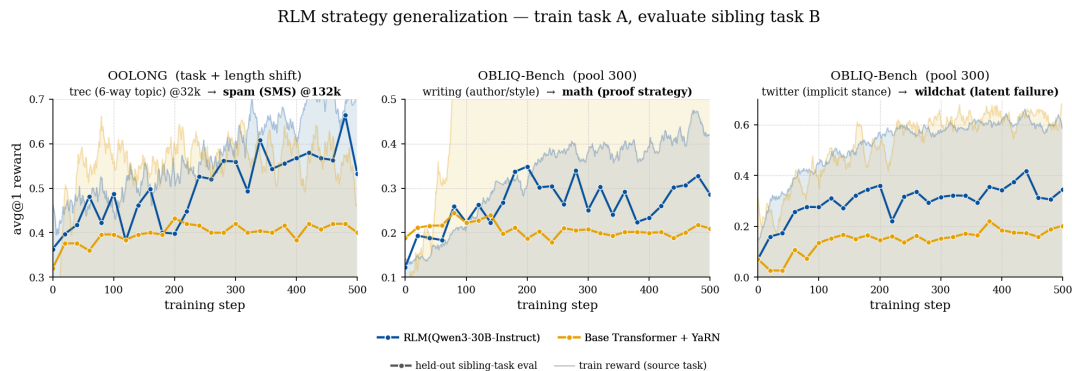


Figure 7. Across three benchmarks with length-varying splits, we train Qwen3-30B-A3B-Instruct-2507 as an RLM and on its own as a Transformer, on one environment domain. We evaluate on a different environment split at every 20 step checkpoint, and plot both the evaluation score and train reward (smoothed) over 500 steps.

We similarly find that the RLM exhibits clear generalization capabilities on **completely different domains** than the one it trained on, while the base Transformer struggles to meaningfully improve. Similar to in the length generalization experiments, initial improvements in eval performance on OBLIQ-Bench during the base Transformer training generally come from learning to follow the proper answer format, which quickly diminishes. Interestingly, *the train reward of the base Transformer generally exceeds that of the RLM despite clear gaps in performance on the eval*. Instead, the train reward of the RLM closely matches the trend of the eval reward, despite being on a completely different domain.

Even more than the length generalization, these experiments have completely different token distributions between train and test, as the main similarity is in the latent task structure that the model must learn to exploit. Like in length generalization, these results suggest the internal mechanisms of the Transformer struggle to decompose the tasks into composable patterns that generalize.

On the cost of training harnesses like RLMs. On similarly-sized tasks, the RLM incurs an extra runtime and memory cost over naively training a Transformer. In particular, across both the length and strategy generalization experiments, the runtime of RLM training is 1.5-3x longer than the base Transformer counterparts due to multiple steps per sample and waiting on sub-calls. However, this cost scales well with task complexity, as training a Transformer on longer context / horizon tasks is significantly more expensive. Even training a simple ReAct agent on the tasks above on an 8xH100 is difficult for a 30B model due to context bloat.

How close are unseen RLM task trajectories to train trajectories?

It is rather difficult to characterize whether a prompt x is “in-distribution” for a language model, but we can use benchmark performance as a proxy for this kind of thing — e.g., LMs can solve most AIME problems, so we consider these tasks to be “in-distribution”. For agent trajectories, which are often long, multiple turns, and filled with extra fluff like tool calls and harness tokens, it is even harder to determine whether two trajectories will exhibit the same output distribution behavior.

If the RLM produces a trajectory on an unseen task that is *token-for-token* identical to a trajectory generated during a training task, then we could easily argue that these two tasks are in the same equivalence class under $\sim_{RLM}$ and are viewed as isomorphic. Unfortunately, even small, one-token perturbations can semantically change the meaning of a text sequence, so arguing that P_x and have some bounded difference is difficult. In the **Appendix**, we discuss some proxies for determining whether two trajectories fall under the same equivalence class. Below, we provide various examples of eval trajectories from our experiments that are similar to some train trajectory on a different task.

← Prev
OOLONG
Next →

1 / 9
token_lcs 0.824
NEARLY_IDENTICAL · 93/100

Train
r=1.00 · 8 turns · lengthgen-oolong-nohints/train@118/57

ASSISTANT

REPL · PYTHON

```
print(f"Context length: {len(context)} character")
print(f"First 500 characters:\n{context[:500]}")
print(f>Last 500 characters:\n{context[-500:]}")
```

Eval
r=1.00 · 8 turns · lengthgen-oolong-nohints/eval@131/23

ASSISTANT

REPL · PYTHON

```
print(f"Context length: {len(context)} character")
print(f"First 500 characters:\n{context[:500]}")
print(f>Last 500 characters:\n{context[-500:]}")
```

OBSERVATION	OBSERVATION
STDOUT Context length: 78138 characters First 500 characters: The following lines contain 787 general-knowledge questions, one per line. Each question has an answer that can be described as one of 6 categories: 'location', 'human being', 'entity', 'abbreviation', 'numeric value', 'description and abstract concept'. You will be asked to answer questions about the aggregate label statistics across all 787 examples in this dataset. Do not try to guess, estimate, or approximate the result. Calculate the exact answer given these datapoints. Date: Apr 18, 2022 Last 500 characters: al name ? Recall: the preceding lines contain 787 general-knowledge questions, one per line. Each question has an answer that can be described as one of 6 categories: 'location', 'human being', 'entity', 'abbreviation', 'numeric value', 'description and abstract concept'. You will be asked to answer questions about	STDOUT Context length: 618252 characters First 500 characters: The following lines contain 6374 general-knowledge questions, one per line. Each question has an answer that can be described as one of 6 categories: 'numeric value', 'human being', 'entity', 'description and abstract concept', 'location', 'abbreviation'. You will be asked to answer questions about the aggregate label statistics across all 6374 examples in this dataset. Do not try to guess, estimate, or approximate the result. Calculate the exact answer given these datapoints. Date: May 21, 20 Last 500 characters: feet ? Recall: the preceding lines contain 6374 general-knowledge questions, one per line. Each question has an answer that can be described as one of 6 categories: 'numeric value', 'human being', 'entity', 'description and abstract concept', 'location', 'abbreviation'. You will be asked to answer questions about

Remark. The ideal behavior is for the root LM to learn to abstract away information about the domain, and isolate decomposable behavior about a task, while deferring domain-specific information to sub-calls. In many cases, the RLM will still choose to print out task-specific information and pass it back to the main context. While this behavior is often undesirable from a generalization perspective (and can likely be trained out if done carefully), we still find that across all experiments, the strategies used by the RLM during evals closely mimic those learned during training. This includes both general, programmatic sub-agent calling strategies like chunking and fanning out sub-calls, as well as general programmatic strategies like `regex` calls.

The Scaling Picture and Moving Forward

It is easy to walk away from these powerful early results thinking that we should all be tinkering with harness designs or imposing our problem-specific intuitions around overly structured programmatic strategies such as MapReduce or dynamic programming. But make no mistake, doing that we will inevitably run afoul of the bitter lesson and fall by the wayside within months.

Instead, we're making a different argument. **Scaling data will remain the biggest driver of progress, but the machinery that we feed that data into and its inductive biases are what will determine the coefficients of that scaling.** And as things stand today, it seems that the returns on scaling Transformers and related neural architectures are just not budging in terms of being poor compositional generalizers. We think this is because their narrow design space of mostly differentiable neural operators misses something fundamental.

But thanks to language as a powerful substrate for which we have been able to train at scale for the past several years, the architecture of our AI systems is no longer limited to simple differentiable operators or to low-level geometric inductive biases. We can now encode far higher-level and more symbolic inductive biases and train our systems end-to-end with RL, and we have shown in this post that through context offloading and programmatic sub-agents, the RLM architecture is one that can be far superior to a vanilla Transformer at generalization, across lengths and domains. And at its core, this all is about an extremely simple but important idea in harness design: make sure these systems can learn to reduce complex problems to sequences of individually and locally in-distribution observations.

In short, better returns on scaling require compositional generalization, and the capacity for compositional generalization looks like it has to largely live in what today we refer to as a harness, but which in the future may blur quite seriously with what we consider the fundamental architecture of our frontier AI systems.

Appendix

Recall that we are interested in characterizing how close harness trajectories on unseen tasks are to what they learned during training. It is rare for two LM trajectories to be token-for-token identical, but taking some liberties with rough smoothness assumptions on the LM as a function of its input, what we really care about is whether two trajectories are close enough under some metric so their output distributions are almost the same.

It is very hard to formalize the above in an empirically meaningful way; we would need to know some properties about the function defined by the LM, a distance metric on input prompts that are within some ϵ -ball, and a distance metric on the output logits that are within some ϵ -ball. As a basic start, we are interested in, for some distance metric $d(\cdot, \cdot)$, the difference between the root LM trajectory of the RLM during evaluation and training. We can define “isomorphic” here loosely as two prompts have some distance less than ϵ , but as mentioned earlier a more rigorous treatment of this topic is likely necessary. We generally just want to understand and find, for this distance metric, what is the average distance of the *closest* training pair to the evaluation rollouts? In other words, at training step t , for trajectories x , the set of evaluation samples $\mathcal{E}_t$, and the set of training samples $\mathcal{T}_{\leq t}$ that occurred before, we compute (with a bit of abuse of notation):

$$\mathbb{E}_{e \sim \mathcal{E}_t} \left[\min_{\tau \in \mathcal{T}_{\leq t}} d(x_\tau, x_e) \right] \approx \frac{1}{|\mathcal{E}_t|} \sum_{e \in \mathcal{E}_t} \min_{\tau \in \mathcal{T}_{\leq t}} d(x_\tau, x_e)$$

We plot the similarity, i.e., of the average eval rollout of the best RLM vs. base Transformer checkpoint to any training trajectory that occurred before it. We consider the following distance metrics:

Plot label	Metric	Distance definition $d(x_{\text{поезд}}, x_{\text{оценка}})$
Edit	Token-level, length-normalized Levenshtein Distance	Let Lev be token-level Levenshtein distance and define normalized similarity
		$\text{LevSim}(a, b) = 1 - \frac{\text{Lev}(a, b)}{\max(\ a\ , \ b\)}$. Then $d(a, b) = 1 - \text{LevSim}(a, b) = \frac{\text{Lev}(a, b)}{\max(\ a\ , \ b\)}$.
Contain	3-gram word containment	Let $N_3(x)$ be the multiset of word 3-grams in x . Then (asymmetric):
Jaccard	3-gram Jaccard	
Weighted Jaccard	Weighted Jaccard	Let $c_x(t)$ be the token (or n-gram) count of type t in x . Then
Length	Length Ratio	(Content-blind) with sequence length $\ x\ $:

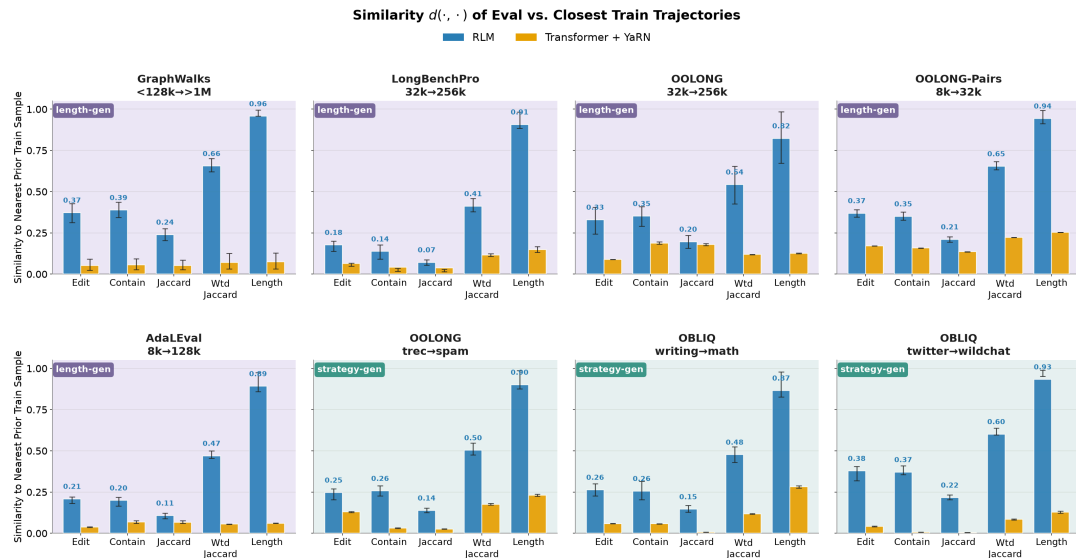


Figure 8. Across the 5/6 length generalization experiments that include a base Transformer and the 3 strategy generalization experiments, we plot the average distance of the eval trajectories of the best training checkpoint to its nearest training trajectory it has seen before. We plot across 5 different distance metrics to highlight how much closer RLM trajectories on unseen tasks look than an appended context baseline like a base Transformer.

Figure 8 illustrates the general point that the root LM’s trajectories share more similarities to the trajectories it sees during training, which primarily comes from context offloading. However, the plots above don’t fully capture the semantic similarities between these trajectories (e.g. it doesn’t say anything about if the RLM generally chooses the same decomposition strategy as its train trajectories).

A more principled distance metric will factor in token similarity and semantic similarity, but it is at least clear from the proxy metrics above that what the RLM’s individual LM calls see, even across a longer time horizon, are much closer to what they may see during training than just a base LM call.

Acknowledgements

Special thanks to Braden Hancock and the Laude Institute for generously providing several 8xH100 nodes on short notice to help run these experiments through the Laude Slingshots program. Alex thanks his wonderful labmates at MIT OASYS and Noah Ziems for helpful feedback while writing the blog.

Citation

You can cite this blog here:

```
@article{zhang2026harnesses,  
  title   = "Language model harnesses are compositional generalizers",  
  author  = "Zhang, Alex and Khattab, Omar",  
  year    = "2026",  
  month   = "July",  
  url     = "https://alexzhang13.github.io/blog/2026/harness/"  
}
```

Footnotes

- 1. Для педантичности отметим, что для того, чтобы эта конструкция класса эквивалентности работала, нам нужно какое-то понятие «эквивалентности», основанное на попадании в эпсилон-шар с центром в какой-то точке пространства траекторий / токенов в соответствии с какой-то метрикой расстояния, а не только на расстоянии между двумя траекториями. Если это кажется вам бессмысленным, просто не обращайтесь внимания, но если вам интересно, как мы можем объединить «похожие» траектории в один класс эквивалентности, не нарушая транзитивности, то вот как это делается. [↗]
- 2. Примечательно, что приведенный ниже аргумент применим рекурсивно, поскольку мы можем рассматривать каждый выгруженный субагент как отдельный экземпляр с собственным «основным» контекстом. Именно такова предполагаемая структура рекурсивной языковой модели, но это не является строго необходимым условием для сохранения локальных наблюдений. [↗]

References

1. **The Mismanaged Geniuses Hypothesis** [link]
Zhang, A., Li, Z. and Khattab, O., 2026.

2. **Exploring Length Generalization in Large Language Models** [PDF]
Anil, C., Wu, Y., Andreassen, A., Lewkowycz, A., Misra, V., Ramasesh, V., Slone, A., Gur-Ari, G., Dyer, E. and Neyshabur, B., 2022.

3. **ReAct: Synergizing Reasoning and Acting in Language Models** [PDF]
Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y., 2023. International Conference on Learning Representations.

4. **Executable Code Actions Elicit Better LLM Agents** [PDF]
Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H. and Ji, H., 2024. International Conference on Machine Learning.

5. **Batch size-invariance for policy optimization** [PDF]
Hilton, J., Cobbe, K. and Schulman, J., 2022. Advances in Neural Information Processing Systems.